A Parameterized Inference Accelerator for Multi-Subject Expression Classification

1. Problem

Neural network inference is dominated by multiply-accumulate operations that are mutually independent within a layer. This makes it a good candidate for an fpga project. The questions I have are:

- How far does parallelism scale before something else binds? Adding multipliers increases throughput only until weight delivery becomes the constraint. Where that transition occurs is a property of the memory organization, not of the arithmetic, and it is measurable.
- What does numerical precision cost in silicon, and what does it buy in accuracy? Reducing precision can lower memory and arithmetic resource requirements and may allow more operations to be packed into the available resources. Because the benefit depends on the Cyclone V DSP architecture and how synthesis maps to it, the actual cost will be measured rather than assumed.
- How does each implementation degrade under increasing load? A processor's cost grows linearly with the number of inputs to classify. FPGA's cost doesn't have to. The crossover is the practical argument for dedicated inference hardware.
- How does worst-case latency differ from average latency, and why?

This project answers all four by direct measurement, comparing a FPGA implementation against the ARM processor on the same chip.

2. Application

The project would be a facial expression classification: given an image region containing a face (or multiple faces), classify it/them into one of a small set of expression categories, displayed as a corresponding emoji. The demonstration runs with two or more subjects simultaneously in frame.

Why this application - The classifier is not the main contribution, because the same multiply-accumulate engine runs any small network with different weights. This application was chosen for three properties:

- Load is a controllable variable. Classifying N faces requires N times the arithmetic. This makes the scaling comparison in Section 5 a good experiment.
- The input rate is fixed by the sensor, giving a hard per-frame deadline.
- The result is an interesting observable demo.

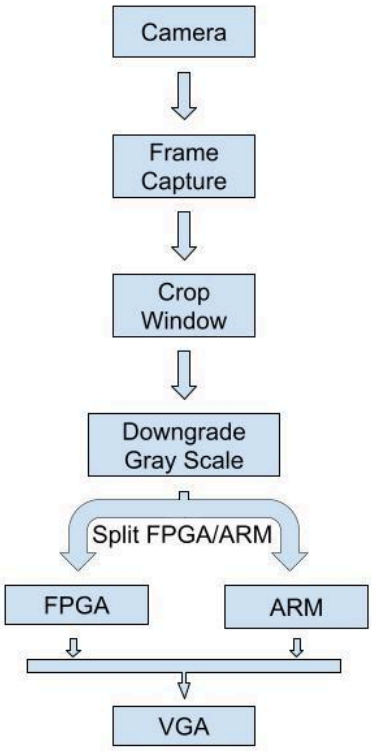
3. System design

3.1. Platform

Terasic DE1-SoC (Intel Cyclone V SoC: FPGA and hard ARM Cortex-A9 processor on one chip), with one camera. (should be same FPGA as 5760)

3.2. Pipeline

The camera image is cropped to fixed regions containing each subject, downsampled to a small grayscale tile, and sent simultaneously to the FPGA accelerator and to C++ running on the ARM. Both produce a classification and a measured execution time. The FPGA additionally reports accelerator kernel latency in clock cycles. The display shows the results alongside the measured latency of each path.



The subjects will stand against a plain light background in known positions and there will be fixed crop windows. The face detection part should be a pre-solved problem. General face detection would be outside the scope of this project.

Both paths receive byte-identical inputs. During measurement, inputs are stored test vectors delivered from the HPS rather than live camera frames, eliminating run-to-run variation. The camera drives the demonstration; the stored vectors drive the experiment.

3.3. The accelerator

The core is a parameterized multiply-accumulate array with on-chip weight storage and a control FSM. Two parameters are exposed and never hardcoded:

- NUM_MACS - parallel multiply-accumulate units
- DATA_WIDTH - bit width of weights and activations

These two parameters are the experiment. Hardcoding either would convert Sections 5.2 and 5.3 from a re-synthesis into a redesign.

Beyond these, the full arithmetic specification of accumulator width, signed representation, rounding versus truncation, saturation behaviour, per-layer scaling, and activation function, will be fixed in the Python reference in week 2, before any RTL is written.

Layer support. Dense (fully connected) layers are implemented first, since they are direct matrix-vector multiplication. Convolutional layers reuse the same MAC array with a different address generation pattern, a change to control and memory addressing rather than to the datapath. This ordering means an end-to-end system exists before the harder layer type is attempted. Convolution is a stretch goal.

Multi-subject handling is a design decision with a measurable tradeoff. Time-multiplexing one accelerator across subjects is area-efficient and scales in time, like software. Replicating the accelerator per subject is faster but expensive in area, and scales in area instead.
Time-multiplexing is the core deliverable; replication is a stretch goal, attempted if the core measurements are complete and area permits. Where both are built, comparing them is a significant part of the technical content.

3.4. Baselines

The FPGA accelerator is compared against two software implementations, both running in C++ on the ARM Cortex-A9 on the same die. The first uses integer arithmetic matching the accelerator exactly, which removes numerical representation as a confounding variable, though the comparison still depends on the quality of the C++ implementation. The second uses float32, the precision the model was trained in. A single comparison would confound the benefit of custom hardware with the cost of reduced precision, and separating them makes each effect measurable on its own. Comparing FPGA against float32 alone would overstate the hardware's contribution, since part of that gap comes from quantization rather than from parallelism. Compiler version, optimization flags, threading configuration, and use of ARM NEON SIMD

will be reported. Preprocessing is excluded from kernel comparisons and included in end-to-end measurements.

4. Why this is a good hardware design problem

A neural network can be implemented in an fpga as a state machine that iterates over weights sequentially. That implementation would be a hand built processor running one program, and would demonstrate nothing about hardware design. The distinguishing test is whether additional chip area results in additional throughput.

The design decisions in this project:

- How many MAC units to instantiate, bounded by available DSP blocks.
- How to organize weight storage so those units are not starved. Memory ports are limited, so a naive design will leave multipliers idle waiting for operands. Deciding what is held on-chip, and how often each fetched weight is reused before being discarded.
- Pipeline depth, which sets clock frequency.
- Numerical precision, trades area against accuracy.

5. Measurements

5.1. Latency distribution

Thousands of inferences per path, reported as median, 99th percentile, worst case, and jitter, not as a mean. I will measure two things separately. Kernel latency is the accelerator alone, counted in FPGA clock cycles, and should be exactly the same on every run. End-to-end latency is measured from the ARM and includes the data transfer and software overhead, so it will vary. Separating them matters because the determinism belongs to the accelerator, not to the system around it. The software baselines are measured end-to-end only. Their distributions should show a long tail from cache misses, interrupts, and scheduler preemption under Linux.

5.2. Throughput against subject count

Inferences per second for both paths at 1, 2, 4, and more subjects, for the time-multiplexed fpga configuration, and for the replicated configuration if it is built. I expect software cost to grow linearly with subject count. Time-multiplexed FPGA should also grow linearly, but with a much smaller constant. If replication is built, its cost should stay approximately flat until area is exhausted.

I will report the subject count at which each path first misses the frame deadline. A small network on a downsampled tile is a modest amount of arithmetic, and optimized C++ on the ARM should keep up at realistic subject counts, so this project does not claim that software fails at the task. Where a breaking point is only reached under scaled load, the scaling will be stated

explicitly rather than the base case being described as marginal. The quantity of interest is what each implementation costs while meeting the same deadline.

5.3. Throughput against parallelism, and against precision

- Throughput as NUM_MACS increases, holding precision fixed, identifying where scaling departs from linear, which locates the memory bottleneck.
- Then for each DATA_WIDTH - 16, 8 bits, classification accuracy, DSP and memory and logic utilization, achieved throughput, and maximum clock frequency. This yields an operational tradeoff surface between precision and hardware cost, measured on silicon rather than estimated from operation counts.

5.4. Timing closure

Maximum clock frequency with the limiting path identified, per configuration.

6. Verification

I'll write the network twice: once in Python using the same integer arithmetic the hardware will use, and once in SystemVerilog. The Python version is the reference, slow and known to be correct. Every hardware stage is checked against it in simulation before reaching the board, and the outputs must match exactly rather than approximately. This matters because a broken classifier still outputs a class, and without a reference I couldn't tell a hardware bug from a bad model.

7. Schedule

The schedule adapted to 12hr/week (needs adapting for weeks with holidays, thanksgiving etc.) The plan is tight and has no reserved slack (for breaks exams or sickness etc.). Week 13 is compressed by Thanksgiving. If earlier weeks overrun, the camera and VGA demonstration is probably the first thing cut. The measurements in weeks 8–12 would be the core deliverable.

Week Number	Rough Bench Mark	Presentation
1: 30/8 - 5/9	- Completed project proposal - Enrollment in student indep project	- Seek professor approval
2: 6/9 - 12/9	- Freeze input resolution and expression classes - Select dataset and train/test split - Freeze network topology - Count parameters and operations per inference - Verify weights fit in available on-chip memory - Establish float32 and 8-bit accuracy baselines	- Show python model
3: 13/9 - 19/9	- Parametrized MAC unit - Test bench	- Show golden values from python are reproduced
4: 20/9 - 26/9	- Single layer -	
5: 27/9 - 3/10	- Full network running in simulation, matching Python exactly	- Show the match
6: 4/10 - 10/10	- Get FPGA-to-processor communication working	- Show hardware matches simulation
7: 11/10 - 17/10	- C++ baselines on the ARM, integer and float	- Show all three paths agree
8: 18/10 - 24/10	- Timing instrumentation; first latency measurements	- Show latency distributions
9: 25/10 - 31/10	- Multiple subjects: time-multiplexed. replicated if area permits	- Show scaling curves
10: 1/11 - 7/11	- Precision sweep - 16, 8 bits	- Show accuracy and cost per width

11: 8/11 - 14/11	- Parallelism sweep; locate the memory bottleneck	- Show where scaling flattens
12: 15/11 - 21/11	- Timing closure; assemble results	- Show Fmax per configuration
13: 22/11 - 28/11	- Camera and VGA display; report draft	- Live demo
14: 29/11 - 5/12	- Revisions, final report	- Final submission

8. AI Disclosure

I will let you know where I use AI in this project. Since AI is becoming a big part of industry I will probably use it when I get stuck, however, the idea of doing this project is learning so I will try to minimize usage. I may use AI for writing summaries, just like I used AI in crafting this document (help with content and formatting). I may also use AI for test cases.